Chapter 2 - Syntax

A) Variable declarations and operators

1. Which of these are valid variable names:
   abc, _under_bar_, 9qdf, m928134, t5, me@var, j7, transient, her_sales$, var, some#, if

2. Will the following compile? Predict and try out.
   a) boolean b = 1;
   b) boolean b = FALSE;
   c) byte b = -128;
   d) char c = 5;
   e) char c = “c”;
   f) char c = -2;
   g) int x;
      char c=’c’;
      x=c;
   h) float f= 5;
   i) float f= 5.0;
   j) double d = 5;
      float f = d;
   k) double d = 5;
      float f = (float)d;
   l) long l = 6;
   m) long l = 123_456_789;
   n) long l = 12345;
      int i=l;
   o) long l = 12345;
      int i=(int)l;
   p) int i = 012; // What is the value of i?
   q) int i= 0x12_E; // What is the value of i?
   r) Try out other things you think of...

3. What is the value of x after each statement is performed?
   a) int x = 7 + 3 * 6 / 4 - 1;
   b) int x = 5 % 3 + 2 * 4 - 3 / 2;
   c) int x = (3 * 9 * (3 - (9 * 3 / (3))));

4. What is the result of following code fragments. Predict and try out.
   a) System.out.println(5+3 == 5*4);
   b) System.out.println(5+(3 == 5)*4);
   c) System.out.println(“5”+(3 == 5)+”4”);
   d) System.out.println(5*3 + “6”);
   e) System.out.println(“6” + 5 + 3*6);
   f) System.out.println(“6” + 5 + 3);
   g) System.out.println(“6” + (5 + 3*6));
   h) byte i = 5; byte j = 6; byte k = i+j;
   i) int i = 5; int j = 6; j+=i;
      int k = i++ + ++j;
      System.out.println(k);
   j) boolean b= false;
      System.out.println(b=false);
   k) System.out.println(5!=4?6:3);
   l) Try out other things you think of...

B) Flow Control

1. **IF-ELSE**
   a)
```
int i = 1;
if (++i == 1) {
      System.out.println(“one”);
} else if (i==2) {
      System.out.println(“two”);
 } else {
       System.out.println(“other”);
 }
```
   b)
```
boolean b = false;
 if (b=true) System.out.println(“true”);
 else System.out.println(“false”);
```
   c)
```
String name = "Lion";
int score = 100;
if (!name.equals("Lion"))
    score = 200;
    name = "Larry";
System.out.println(name + “ has score “ + score);
```
   d)
```
int i=0;
if (i=10) {System.out.println(“ten”);}
```

2. **SWITCH**

a) short x = 5;
```
switch (x) {
  case 4: System.out.println("four");
  case 5: System.out.println("five");
  case 6: System.out.println("six");
  default: System.out.println("other");
}
```

b) String x = "5";
```
switch (x) {
  case 4: System.out.println("four");
  case 5: System.out.println("five");
  case 6: System.out.println("six");
  default: System.out.println("other");
}
```

c) String x = "five";
```
switch (x) {
  case "four": System.out.println("four"); break;
  case "five": System.out.println("five"); break;
  case "six":  System.out.println("six"); break;
  default: System.out.println("other"); break;
}
```

d) String x = "five";
```
switch (x) {
  case "four": System.out.println("four"); break;
  default: System.out.println("other");  break;
  case "five": System.out.println("five"); break;
  case "six": System.out.println("six"); break;
}
```

e) short x = 5;
```
int i = 6;
switch (x) {
  case 4: System.out.println("four");
  case 5: System.out.println("five");
  case i: System.out.println("six");
  default: System.out.println("other");
}
```

3. **WHILE / DO WHILE**

a) int i = 5;
```
while (i<=10){
  System.out.println(i);
   i+=2;
 }
```

b) int i = 5;
```
do {
  System.out.println(i);
   i++;
 } while (i>6);
```

c) int i = 5;
```
while (i<=10)
   System.out.println(i);
   i+=2;
```

d) String s = “b”;
```
do {
  System.out.println(“not b”);
  s=”c”;
 } while (s = “b”);
```

4. **FOR**

a)
```
for (int i=0; i<=5;i+=2)
    System.out.println(i);
```

b)
```
for (int i=0; i<=5;)
   {System.out.println(i);
    i+=2; }
```

c)
```
for (int i=1; i<3; i++){
   for (int j=5; j>0; j--) {
     System.out.println(i + “ ” + j);
   }
  }
```

d)
```
for (int i=1; i<3; i++){
   for (int j=5; j>0; j--) {
     if (i==j) break;
     System.out.println(i + “ ” + j);
   }
  }
```

e)
```
for (int i=1; i<3; i++){
   for (int j=5; j>0; j--) {
     if (i==j) continue;
     System.out.println(i + “ ” + j);
   }
  }
```

5. **COMBINATIONS**

a)
```
public class Test {
    public static void main(String[] args) {
        outerloop:
        for (int i=0; i < 3; i++) {
            for (int j=0; j < 3; j++) {
                if (i * j > 4) {
                    System.out.println("Breaking");
                    break outerloop;
                }
                System.out.println(i + " " + j);
            }
        }
        System.out.println("Done");
    }
}
```

b)
```
public class DoBreak1{
    public static void main(String[] args){
        String[] table = {"aa", "bb", "cc", "dd"};
        for (String ss: table){
            if ( "bb".equals(ss)){
                continue;
            }
            System.out.println(ss);
            if ( "cc".equals(ss)){
                break;
            }
        }
    }
}
```

C) Arrays

1. Find and correct the error in each of the following program segments:
   a) int[] b = new int[10];
      for (int i = 0; i <= b.length; i++)
         b[i] = 1;
   b) int[][] a = {{1, 2}, {3, 4}};
      a[1, 1] = 5;

2. What are the results of following code fragments? Predict and try out.
   a) String[] a = new String[3];
      a[1] = “this”;
      a[2] = “is”;
      for (String s: a) {System.out.println(s.toUpperCase());}
   b) String[][] a = {{“this”,”is”,”an”},{“array”}};
       System.out.println(a[1][0]);
   c) String a[] = new String[3];
      a[0]=”el1”;
      a[1]=”el2”;
      System.out.println(a.length());
   d) int[] arr[] = new int[2][];
      arr[0]=new int[]{1,2,3};
      arr[1] = {5,2};
      System.out.println(arr[0][1]==arr[1][0]);
   e) int[] arr1 = {1,2,3};
       int[] arr2 = arr1;
       arr2[1]=4;
      for (int i=0; i<arr1.length;i++) { System.out.println(arr1[i]); }

D) String

1. What are the results of following print statements? Predict and try out.
   For all cases: String s = “testing java strings”;
   a) System.out.println(s.charAt(1));
   b) System.out.println(s.append(“ 123”));
   c) System.out.println(s.indexOf(“s”,2));
   d) System.out.println(s.equals(“testing java Strings”));
   e) System.out.println(s.substr(3,6));
   f) System.out.println(s.concat(“again”).toUpperCase().replace(“T”,”o”).substring(4,7));

2. What are the results of following code fragments? Predict and try out.
   a) String s = “hello”;
      s.concat(“ again”);
      System.out.println(s + “ my friend”);
   b) String s= “hello”;
      s+=” again”;
      System.out.println(s.length());
   c) String s= “hello”;
      s+=” again”.toUpperCase();
      System.out.println(s.reverse());

F) General exercises

1. Create an array of integers. Calculate the average of those numbers.
2. Create an application that:
   - Reads a language code that the user has typed in (Look for the Scanner class)
   - Displays a welcome message in the given language
   - If the language code is not recognized, display a message with multiple greetings

   Examples of input and expected result:

   "NL" : Goeiedag!

   "FR" : Bonjour!

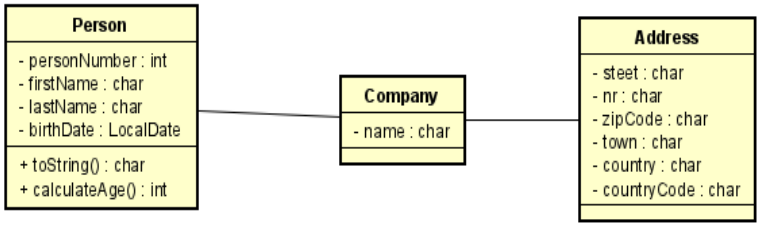
   "EN" : Good day!

   Other : Goeiedag, bonjour and good day!
3. Create an application that:
   - greets the user with "Welcome!"
   - asks the user: would you like to be greeted again? (Y/N)
   - reads the answer and if
     - Y: welcomes the user again and repeats the question
     - N: ends the application

Chapter 3 - Building Blocks

A) ABIS Client and Course Administration (ACCA)

1. Create a new Java project called *ACCA*. Create 2 packages: *be.abis.acca.model* and *be.abis.acca.test*.
   Check out the following UML diagram. Implement this (note: char is to be implemented as String in Java). Make sure to provide constructors + getters/setters. Replace the birthDate by an int age for the moment, and make the calculate age just return the age. Use a test class to create 2 persons, one with a company linked with him, and one without.



2. Add a *printInfo()* method in the Person class, which prints out more detailed information of each person.
   E.G. “Person 1: Ann Smits (35 years old) works for ABIS in Leuven” , “John Doe (54 years old) is not employed for the moment”. Put the created persons in your test class in an array, and loop over it, calling the method.

3. Add a *Course* class in the model. Each course has a title, a number of days, a price per day and a field indicating whether prior knowledge is required to follow the course.
   Choose correct names and data types for these variables.

4. Add a method *calculateTotalPrice()* in the Course class. This mainly multiplies the duration and price per day of the course. Also add a VAT of 21%. but only when the course takes less than 3 days, and prior knowledge is not required.

5. We want to label the courses based on the total price:
   < 500 : “CHEAP”
   between 500 and 1500 : “OK”
   > 1500 : “ EXPENSIVE”
   Add a method *labelPrice()* in the Course class to do this

6. Create a method *printInfo()* which prints the info for a specific course.
   E.G. “The *Java* course takes *5* days. Prior knowledge is *not* required. The total price is *2123.45*. This is *expensive*.”

B) Palindromes

1. Create an application that handles palindromes. A palindrome is a word, phrase, number, or other sequence of characters which reads the same backward or forward. For example: “radar”, “racecar”, “wow”, “tacocat”.

   - Create a class Palindrome, with a method *boolean isPalindrome(String word)* that checks whether a passed word is a palindrome or not.
   - Add a method *int countNumberOfPalindromes(String[] palindromes)* that counts how many of the words passed in an array, are really palindromes. We won’t accept one letter words as correct palindromes.
   - Create a main application to check for existing palindromes
     - Create an **array** of 'possible' palindromes starting from Strings
       e.g. kok, pap, lepel, droomoord, legerregel, level, *null*, negen, error, lol, meetsysteem, legovogel, parterre-trap, tarwewrat, kaak, a, racecar
     - Loop through the array. Print out all correct palindromes + how many you have found

Chapter 5 - Inheritance and Polymorphism

A) ABIS Client and Course Administration (ACCA)

1. Add a class AbisEmployee as subclass of Person. As extra fields, the AbisEmployee should have a salary and a home address. Override the *printInfo()* method, such that it also prints the salary and home town.

2. Integrate the salary handling ideas of Java introduction and syntax (exercise B1 in the Java Programming course).

3. An AbisEmployee can play one or more roles, e.g. instructor, manager, accountant, sales, ... Add an interface *Role*, with the different possibilities as implementation classes.

4. Add an *AbisEmployee[]* in the Course class.
   Make sure that the added employee has the role of instructor. Print the names of the instructors in the *printInfo()* method.

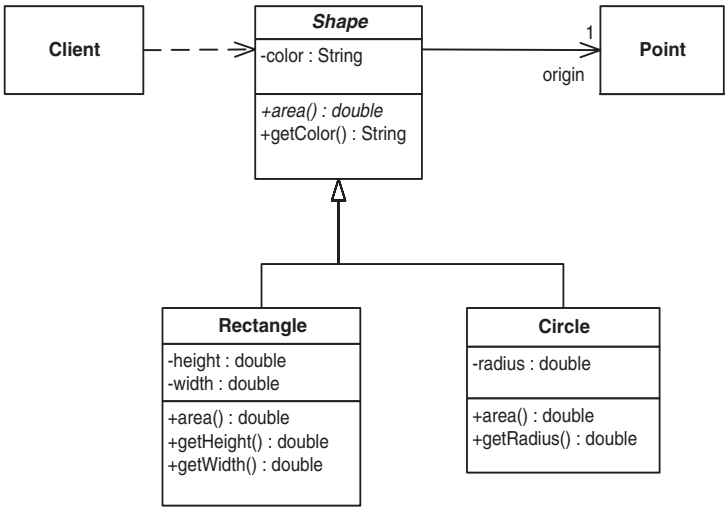
B) Palindromes

1. Modify the Palindrome application in the following way:

   - create a subclass **PalindromePhrase** of **Palindrome** containing phrases
     e.g. "Never odd or even" or "A Toyota's a Toyota" or "Was it a car or a cat I saw?"
   - override the *isPalindrome()* method, which will eliminate spaces or punctuation marks of the returned String.
   - add a toString() method to the Palindrome class
   - in the main, create an array of type Palindrome, which can contain both objects. Do the same tests as before

C) Shapes

1. Program the following situation. Implement the *area()* method.



2. Provide the polymorphic area() method, which calculates the area of the shape .

3. In the client (application) class, create an array of Shape objects, and print their areas.

4. Add a toString() method.

5. Make sure to also print the color and the dimensions of each Shape.

6. Add classes Square, Line, Triangle, ...

7. Provide enumeration Color instead of String, for the attribute color in the Shape class.

Chapter 6 - Exceptions

A) ABIS Client and Course Administration (ACCA)

1. Create an exception *PersonShouldBeAdultException*, which is thrown from the calculateAge() method if the person is younger than 18.
   Also add a *PersonShouldNotBeRetiredException*, which is thrown from the same method, if the person is over 67 years old.
   Create a superclass *AgeException* of previously created exceptions.

B) Palindromes

1. Throw a *NotAPalindromeException* when the tested value is not a palindrome.

C) Shapes

1. Add a method checkArea(), which throws an AreaTooBigException when the area superseeds a certain value.

Chapter 7 - Collections

A) ABIS Client and Course Administration (ACCA)

1. Replace all arrays in the model classes by ArrayLists.
2. Make it possible to sort persons by personNumber, age or first name.

B) Palindromes

1. Switch the array used into an Set<Palindrome>.
2. Sort the palindromes alphabetically.

C) Shapes

1. Add the shapes into an List<Shape> in the Test class.
2. Provide methods
   - compareArea() to check whether areas are equal, taller, or smaller
   - resize() to change the size of an object (e.g. double the area - preserve aspect ratio)
3. Sort the shapes by area.

Chapter 8-10 - Utilities, I/O and Lambdas

A) ABIS Client and Course Administration (ACCA)

1. Change the int age to LocalDate birthDate everywhere. Make sure to implement the calculateAge() method correctly.

2. Print the total price of a course in format 1234,56.

3. Create a file called addresses.csv, which contains all addresses. The different fields should be seperated by ";".
   Add an AddressRepository interface in a package be.abis.acca.repository, with an implementation class called FileAddressRepository.
   Add methods findAllAddresses() and insertAddress() that respectively read the addresses into a List<Address>, or appends a new address to the file. Attach this directly into the main and test.

4. Add a method findAddress() to previous class, that searches an address based on street, nr and zipCode. Implement this by looping through the list of addresses.

5. Create an overloaded method of findAddress(), which only takes in the nr and zipCode. Implement this by using streams.

6. Make sure that you can sort the addresses by zipCode or by countryCode and town. Use lambdas to achieve this.

B) Palindromes

1. Read possible palindromes from a file.

2. Print all correct palindromes, sorted alphabetically, to a new file.

3. Add a *PercentageChecker* interface (implemented via a lambda), which tests whether more than a certain percentage (passed as variable) of the list values are palindromes.

C) Shapes

1. Move the checkArea() method to an AreaTester class.
   Add a Predicate<Integer> to the checkArea() method (implemented via a lambda of course), which tests whether a shape has an area of less than 1 cm² (passed as variable).

2. Remove the 'small' shapes from the collection.